05 NOV 2026 / RETO 02

Optimización y producción

Informe autónomo de PharmaSoft para Lenguaje de Programación II. Las mediciones se hicieron sobre la actividad práctica de la sesión 13, Angular en Chrome, Spring Boot en perfil prod, Oracle aislado y Redis en Docker.

Frontend · pharma-frontend · PR #3

Backend · PharmaBackend · PR #1

Resultados principales

Indicador	Referencia	Versión evaluada
Paquete inicial (bruto / estimado)	309.74 kB / 87.73 kB	303.65 kB / 85.85 kB
Archivos diferidos	27	28
Lecturas de categorías	20	19 aciertos / 1 fallo
Lecturas de productos	20	19 aciertos / 1 fallo
Invalidación y recuperación	6 casos definidos	6 casos comprobados
DNI de cliente de prueba en log	1 coincidencia	0 coincidencias

La versión final carga el layout autenticado después de pasar el guard, conserva la navegación bajo demanda y muestra el código de seguimiento cuando el backend devuelve un error. También se retiró del log el DNI que se registraba al crear clientes.

Alcance de las mediciones

Los tiempos son observaciones locales, con datos de demostración y sin latencia de una sucursal remota. Se usaron los mismos pasos en cada comparación. Las secciones A a D explican cómo repetirlos y la sección E describe la puesta en marcha.

A / TAMAÑO DEL PAQUETE

Carga diferida

Se ejecutó ng build --stats-json con configuración de producción. Se sumó bytesInOutput de cada módulo en los JS del bloque Initial chunk files, excluyendo los chunks diferidos.

```
browser-stats.json | 5 módulos con más bytes en el JS inicial
1   136 281 B  @angular/core / _debug_node-chunk.mjs
2    87 294 B  @angular/router / _router-chunk.mjs
3    35 203 B  @angular/core / _pending_tasks-chunk.mjs
4    25 076 B  @angular/common / _module-chunk.mjs
5    21 665 B  @angular/common / _common_module-chunk.mjs
```

A-analyzer.png · ranking reproducible calculado del metafile de esbuild.

N.º	Módulo	Bytes	Origen
1	@angular/core / _debug_node-chunk.mjs	136 281 B	Angular · core
2	@angular/router / _router-chunk.mjs	87 294 B	Angular · router
3	@angular/core / _pending_tasks-chunk.mjs	35 203 B	Angular · core
4	@angular/common / _module-chunk.mjs	25 076 B	Angular · common
5	@angular/common / _common_module-chunk.mjs	21 665 B	Angular · common

```
Angular CLI | misma compilación de producción
Antes:   309.74 kB / 87.73 kB   | 27 archivos diferidos
Después: 303.65 kB / 85.85 kB   | 28 archivos diferidos
chunk-BBktYeja.js | main-layout | 6.34 kB | 2.20 kB
```

A-build.png · salida resumida de las compilaciones reales.

El layout era un import estático en app.routes.ts. Se migró a loadComponent dentro de la ruta autenticada. El login conserva su ruta independiente. Salieron 6,09 kB brutos y 1,88 kB de transferencia estimada del paquete inicial.

A / COMPARACIÓN DE PRECARGA

Primera pantalla y Ventas

Tres repeticiones por variante en contextos nuevos de Chrome con caché deshabilitada. Se midió el login en frío y se abrió Ventas 1,2 segundos después de que apareció su enlace en Inicio.

Mediana	Bajo demanda	PreloadAllModules
Primera pantalla de login	149 898 B	200 908 B
Login y ventana de Inicio (total)	174 010 B	219 198 B
Desde clic en Ventas al título	62.52 ms	50.48 ms
Desde enviar login hasta Ventas	1426.84 ms	1401.03 ms

Captura de la red de Chrome · tres repeticiones

```
Sin precarga: 131.27 ms, 56.48 ms, 62.52 ms
Con precarga: 50.48 ms, 49.95 ms, 54.05 ms
```

La precarga empezó incluso en la pantalla de login. Aumentó el total transferido antes de abrir Ventas en 45 188 B y redujo la mediana del clic a Ventas en 12.04 ms. Los tiempos de muestras individuales se solapan.

Se mantuvo la carga bajo demanda para los tres locales. La reducción de tráfico inicial favorece a la sucursal de conexión más lenta. Una prueba remota con la latencia y el ancho de banda reales permitiría evaluar la precarga selectiva.

B / ENCABEZADOS HTTP

Políticas de caché

Se comparó curl -I con la recarga en Chrome sobre Nginx. Los archivos con hash ya estaban en la caché después de la primera visita.

Recurso	Cache-Control (curl y Chrome)	Recarga	Correcto
/index.html	no-cache	red / HTTP 200	Sí
main-WWJJ7G7Y.js	public, max-age=31536000, immutable	memoria	Sí
chunk-DT3yjk5a.js	public, max-age=31536000, immutable	memoria	Sí
styles-4W23OAWM.css	public, max-age=31536000, immutable	memoria	Sí
/api/v1/categorias	no-cache, no-store, max-age=0, must-revalidate	red / HTTP 200	Sí

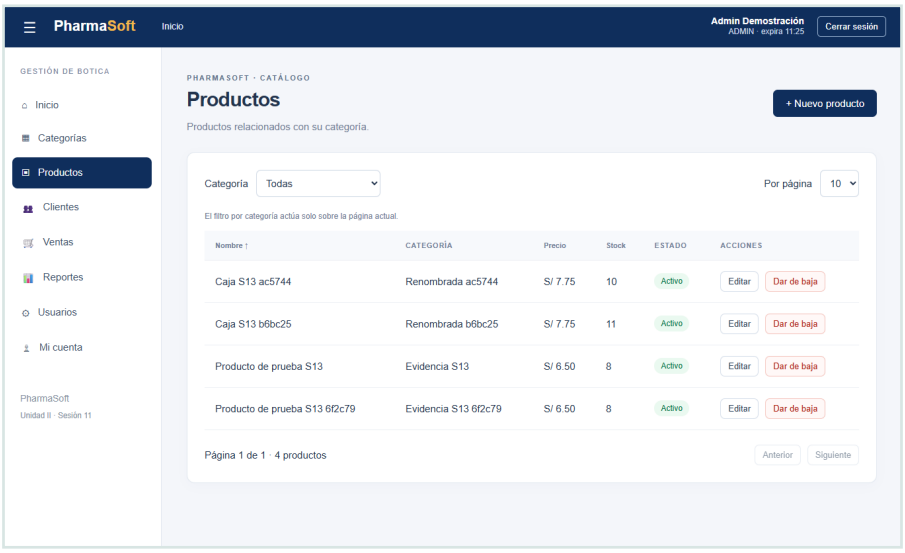
Nginx devuelve no-cache para el HTML, de modo que el navegador verifica su vigencia antes de usarlo. Los nombres con hash permiten conservar el JS y CSS durante un año. En la muestra, el HTML llegó por red con HTTP 200 y los estáticos se reutilizaron desde memoria.

Las respuestas de la API pueden incluir datos privados y existencias que cambian con cada venta. Spring Security devolvió no-cache, no-store, max-age=0, must-revalidate. no-store impide guardar la respuesta. Redis opera en el backend, con invalidación y vigencia propias.

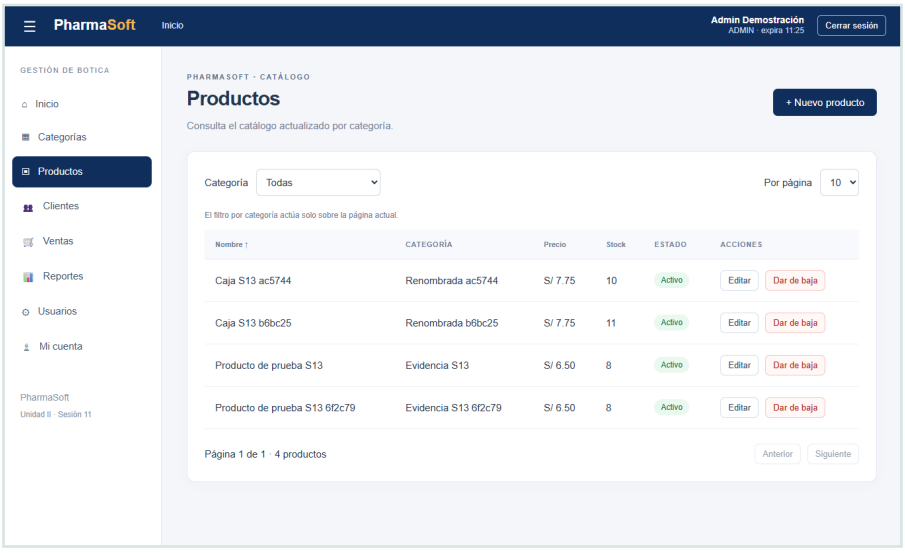
B / SIMULACIÓN DE DESPLIEGUE

Actualizar con Nginx activo

Se cambió el subtítulo de Productos en una compilación temporal. Se copiaron primero los archivos nuevos con hash y se reemplazó el índice al final, manteniendo el contenedor activo.



B-antes.png · texto original en Productos.



B-despues.png · el texto nuevo aparece al recargar sin borrar la caché.

Chrome · JS transferidos después de cambiar el índice

```
main-UNOFRXKQ.js
chunk-CPyKW07p.js
chunk-BkUTlXlH.js
chunk-DYxHrB7P.js
chunk-zkL_zkxX.js
```

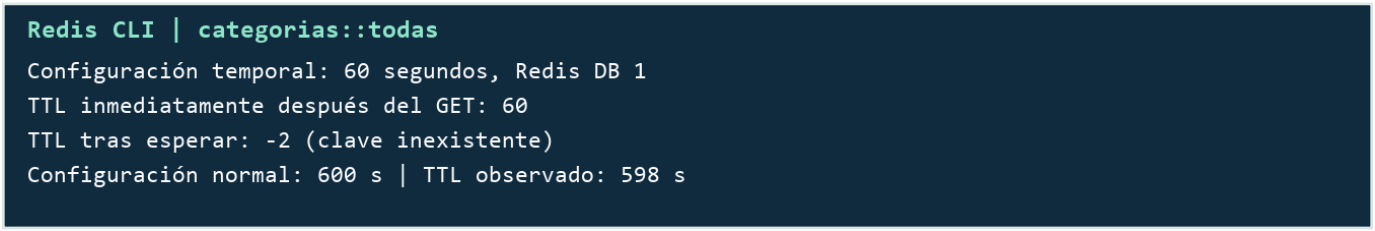
Cambiaron 12 nombres JS al recompilar, incluidos el archivo main y chunks dependientes. Chrome solicitó los 5 JS nuevos necesarios para Productos. Los nombres que se mantuvieron pudieron reutilizarse. Se restauró el build final.

Aciertos, fallos y vigencia

Tras reiniciar el backend se despejaron las claves de la caché evaluada y se hicieron 20 GET por ruta. Se consultó cache.gets con los tags name y result para cada catálogo.

Caché	Aciertos / fallos	Tasa	Primer GET	Media de 19 aciertos
categorias	19 / 1	95 %	141.41 ms	29.06 ms
productos	19 / 1	95 %	72.3 ms	21.21 ms

La tasa se calculó como aciertos / (aciertos + fallos). Cada primera lectura consultó Oracle y guardó la respuesta. Las siguientes reutilizaron la entrada. La autenticación sigue validando al usuario y los permisos, así que Redis reduce las consultas del catálogo y no elimina la comunicación con Oracle.



C-ttl.png · TTL capturado con redis-cli en una base temporal de Redis.

La instancia temporal usó 1 minuto y una base Redis distinta. El resultado -2 confirma que la clave expiró. El backend principal mantuvo 10 minutos y se comprobó su TTL normal en la clave de categorías.

C / MATRIZ DE INVALIDACIÓN

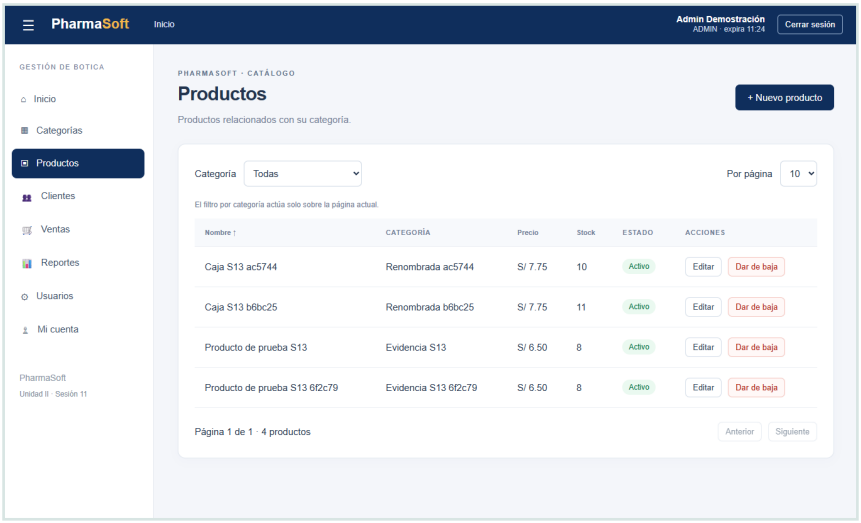
Datos después de cada cambio

Se calentaron las lecturas relevantes antes de cada escritura. La prueba usó una categoría, un producto y un cliente de demostración en el esquema Oracle aislado.

Caso	Resultado observado
C-01	Alta HTTP 201: Auditoria b6bc25 visible en el GET siguiente
C-02	Auditoria b6bc25 -> Renombrada b6bc25, visible en productos tras PUT
C-03	Detalle y lista: S/ 5.50 -> S/ 7.75
C-04	Venta desde la SPA, HTTP 201. Stock 12 -> 11 en lista y detalle
C-05	DELETE HTTP 204 y la categoría sin productos desaparece de la lista
C-06	SPA lista desde Oracle, HTTP 200. Actuator devuelve HTTP 503 DOWN



C-04-stock.png · fila de Productos tras vender una unidad desde la SPA.



C-06-oracle.png · Productos cargó con Redis detenido. Actuator informó DOWN.

Las seis comprobaciones pasaron. El cambio de nombre vació las dos cachés, la edición de precio refrescó lista y detalle, y el registro de la venta invalidó productos. Las lecturas recurrieron a Oracle durante la caída.

D / TRAZABILIDAD

Venta y errores de la SPA

Se registró una venta desde el formulario de PharmaSoft. Chrome capturó X-Request-Id de la respuesta HTTP 201 y se filtró el archivo rotativo por ese valor.

Log de producción filtrado por X-Request-Id

```
2026-11-05 10:54:34,932 INFO  [267d47ad-4459-4f5e-a2ca-9efcf913d351]
p.e.u.P.controller.VentaController - Venta registrada | id=5 | items=1
2026-11-05 10:54:34,939 INFO  [267d47ad-4459-4f5e-a2ca-9efcf913d351]
p.e.u.P.config.RequestIdFilter - POST /api/v1/ventas -> 201 (90 ms)
```

D-venta-log.png · ambas líneas contienen el mismo X-Request-Id.

La línea del controlador confirma el identificador de venta y el número de ítems cuando el servicio ya ha confirmado la transacción. La línea del filtro informa método, ruta, estado y duración de la petición. Para reconstruir el flujo basta el código de seguimiento.

Venta no encontrada con id: 99999 Código de seguimiento: 7da3cb87-02f4-4fbe-b893-e7a2c527e437

D-error.png · consulta de una venta inexistente y código mostrado al usuario.

Mismo error 404 en el archivo de producción

```
2026-11-05 10:54:38,528 INFO  [7da3cb87-02f4-4fbe-b893-e7a2c527e437] p.e.u.P.config.RequestIdFilter -
GET /api/v1/ventas/99999 -> 404 (23 ms)
```

El manejador toma requestId del MDC y lo serializa en el DTO. La SPA muestra el código junto al mensaje. Soporte puede buscar el incidente en el log sin pedir credenciales ni el contenido de la petición.

Salud, métricas y datos privados

Se consultó Actuator sin credenciales y con un usuario administrador, antes de detener Redis y durante la caída.

Escenario	Sin token	Oracle (admin)	Redis (admin)
Redis activo	UP	UP	UP
Redis detenido	DOWN	UP	DOWN

http.server.requests | tag uri:/api/v1/ventas

```
COUNT: 1
TOTAL_TIME: 0.0898954
MAX: 0.0898954
```

COUNT cuenta las solicitudes de esa ruta durante la ejecución del proceso. TOTAL_TIME es el tiempo acumulado en segundos y MAX el mayor tiempo de la ventana del medidor. El estado público informa disponibilidad y el administrador puede identificar el componente que falló.

Auditoría de secretos

Antes de la corrección, el alta de cliente escribía el DNI completo. Se cambió a un mensaje sin ese dato. Después se reinició con un log limpio y se repitieron el alta y la venta. Hubo cero coincidencias de contraseñas, tokens, clave JWT y DNI completos. La búsqueda excluyó los UUID de seguimiento, que pueden contener ocho dígitos por coincidencia.

Despliegue y reversión

Los dos README contienen el diagrama con puertos, variables de entorno, comandos, estado de las buenas prácticas y plan de reversión.

Arquitectura

Los tres locales acceden por HTTPS 443 al proxy TLS propuesto. Nginx sirve Angular en el puerto 80 del contenedor, publicado en 8081. La API escucha en 8080 y consulta Oracle por JDBC 1522 en la práctica local y Redis por 6379 en red privada.

Reversión

Conservar el índice y los activos del build anterior, el JAR, la configuración y un respaldo de Oracle. Restaurar el índice y el JAR compatibles. Invalidar claves si cambia la serialización. Repetir sondas y pruebas de humo.

1. Seguridad de acceso

Publicar un único dominio HTTPS. Restringir Oracle, Redis y Actuator administrativo. Gestionar secretos de Oracle y JWT mediante variables de entorno.

2. Continuidad de ventas

Respaldar Oracle y ensayar la restauración. Compartir Redis entre instancias del backend para que la invalidación de stock sea común.

3. Alertas y recuperación

Medir disponibilidad desde cada local. Alertar por readiness, latencia y errores. Etiquetar cada despliegue y documentar la reversión. Incorporar migraciones versionadas y probarlas con una copia del esquema.

Versionamiento

Los cambios se publicaron en dos repositorios, con varios commits por componente y un PR hacia develop en cada uno. Los commits del Reto 02 corresponden al mismo día real de trabajo. El criterio de tres días distintos queda pendiente.

Ficha de la actividad autónoma

Formato de análisis de esbuild